Trust Is Not Transitive: Verifying the Composition of LLM Guardrail Hierarchies

Majid Babaei

***Abstract*—Guardrails for LLM agents are stacked: deterministic rules, classifiers, LLM judges and human approvers, each relying on what the others did. A stack is usually justified by measuring its parts, yet each part's promise holds only under assumptions, and an assumption that held for one stage says nothing about the next. We give a formal account of when such a composition is justified. A small calculus models hierarchies as gates, guards and routing switches, with guards constrained only by their declared contracts. A credited gate can establish a fact that discharges a later stage's assumption, while a guard's verdict carries no evidence forward. We prove that the resulting composition check is sound, and complete relative to contracts and freely behaving stages: an accepted hierarchy satisfies its goals under every implementation of its contracts, and a rejected one has an execution in which every stage keeps its contract and a violating action is accepted. We prove that reasoning with transitive evidence is unsound, show how stage order affects the guarantee, and show that the exact problem is coNP-complete in general but near-linear when no facts flow between stages; our checker uses the fast analysis and refines its rejections exactly. On these results we build route-level statistical certificates with incremental recertification. Inductive proofs are discharged by two SMT solvers and hardened with vacuity checks and proof mutants; algorithmic and statistical results have paper proofs. The implementation agrees with a model search and reference semantics in bounded-exhaustive and differential tests. The approach does not show that any guard meets its contract. It lists the claims an assurance argument rests on and tests which are needed. In the public example configurations of one framework, stacks are short chains, most of them entirely statistical, and for those the list of claims is the main result.**

***Index Terms*—LLM agents, guardrails, formal verification, assume-guarantee contracts, dataflow analysis, runtime enforcement, statistical certification.**

I. Introduction

LARGE language model (LLM) agents act: a coding agent reads an issue, edits files, runs tests and posts results to a web service [1]. Text the agent reads as data can steer what it does next [2], [3], so production systems place guardrails between the agent and its tools [4], [5]. A guardrail stack may combine pattern rules, classifiers, LLM-based checks and human approval. Some stages can only stop a request; in others, a doubtful case is passed to a costlier stage, as in Trust or Escalate [6] and Conformal Arbitrage [7]. We call any such arrangement a *guardrail hierarchy*.

Stacks are usually assessed by measurement. LlamaFirewall, for example, reports attack success and utility on AgentDojo [3], [4]. Measurement says what happened on the inputs tried. An engineer changing a stack also needs answers about its structure. If a judge now reads a different input, does the old evaluation still apply? Which stages must be reassessed after a scorer is retrained? Does the order of two stages matter? On what does the claim "this action is safe" finally rest?

Each stage's promise is conditional, and that is where stacks go wrong. A stack can accept an unsafe action although every stage meets its contract, because a guarantee was relied on where its assumption had not been established (Section II-A gives an example). Compositional reasoning excludes this step [8], [9], and the same lesson is old in security protocols [10]. In a guardrail stack the step is usually implicit.

The gap. Recent work specifies and enforces agent behavior with temporal rules and contracts [11]–[13], isolates untrusted data by design [14], [15], or gives statistical guarantees for particular cascades [6], [7]. A position paper proposes layered probabilistic assume-guarantee reasoning for agents [16]. We know of no account that gives what software engineers expect of a composition method: a language in which general hierarchies can be written, a formal semantics, a decision procedure proved sound and complete against it, and an implementation shown to conform.

Our approach. We model a hierarchy as a term of a small calculus in which guards are constrained only by their contracts. Evidence for a contract belongs to one invocation of a stage. There is one way for evidence to move forward: a gate whose contract applies can *provide* a fact that later stages may use. A guard's verdict provides nothing. From these rules follows a composition condition, which a checker decides and explains with witnesses.

Contributions.

1) A *calculus of guardrail hierarchies* with a semantics that quantifies over all contract-respecting behaviors, and a specification language, HSL, for acyclic hierarchies (Section III).
2) A *composition condition* with proofs of soundness and of completeness relative to free stages; a theorem that transitive reasoning about evidence is unsound; a result on stage order; and a complexity classification: coNP-complete in general, near-linear without flow of facts, and coNP-hard with correlated conditions (Section IV).
3) *Route-level risk accounting*: risk as a sum over acceptance routes, certificates bound to the identities of a route's prefix, and recertification limited to what a change can reach (Section V).
4) *Guidance for practice*: a worked example, a catalog of contracts by guard class, and a deployment process (Sections VI and VII).

Affiliation and contact information to be added.
Manuscript received ; revised .

5) *Checked proofs and an evaluated implementation* (Sections VIII–X): SMT-discharged inductions, conformance evidence, measurements of scale, and an analysis of public configurations and published designs.

Research questions. RQ1: Are the theorems proved, and do the proofs depend on the definitions? **RQ2:** Does the implementation conform to the definitions, and are its rejections real? **RQ3:** How do checking, deciding and recertifying grow with size? **RQ4:** What do real configurations look like, and what does the approach report on them and on published designs?

Scope. We ask whether a composition is justified given its parts' contracts. We do not ask whether any LLM meets a contract, and we make no claim that any agent became safer. No LLM is used in the study: a theorem over all contract-respecting guards needs no sample of them. Against an adaptive attacker a statistical contract may simply be false, and Table I states what then remains.

II. Background

Guardrails. Guardrails intercept the proposed actions of a tool-using agent. They include deterministic rules, classifiers for injection and harmful content [17]–[19], LLM-based checks of an action against the user's goal [4], [20] and human approval. The central threat is *indirect prompt injection*, in which text read as data contains instructions the agent follows [2]. Guardrail products can be evaded [21]. A guard that reads attacker text can be addressed by it, which is why the *dual-LLM pattern* separates a privileged LLM that never reads untrusted text from a quarantined one with no tool access [22].

Monitors and information flow. A reference monitor mediates every access and is small enough to verify [23], [24]. Execution monitors enforce safety properties of traces [25]. Information-flow control attaches labels to data and propagates them through derivations [26], [27].

Assume-guarantee contracts. A contract (A, G) states that if the environment satisfies assumption A, the component guarantees G [28], [29]. Compositional proof rules [8], [9], [30] are sound only when every assumption is established by the environment or by another component's guarantee. Probabilistic forms of contracts exist [31], [32].

Dataflow analysis. A forward *must* analysis computes the facts that hold on every path to a program point. For distributive transfer functions the iterative solution equals the meet over all paths [33], [34]; on an acyclic graph one pass in topological order suffices.

Statistical certification. For a fixed policy and N independent samples with k losses, the Clopper–Pearson bound is an exact one-sided upper confidence bound on the loss probability [35], and it is used in statistical model checking [36]. Bounds at level δ/m hold simultaneously for m quantities by the union bound [37]. These guarantees assume independent or exchangeable data [38].

A. Running Example

DevAgent triages issues, edits files and calls HTTP services. Deterministic gates check production approval (φ_1), secret egress (φ_2) and the execution environment. Two semantic goals remain: φ_3, never act on instructions embedded in untrusted content; and φ_4, the change does what the requester asked. Four traces recur in this paper. In T^*, a malicious issue tells the agent to upload a secret file to an external host. In T_b, a benign issue asks for public diagnostics to be attached to a local report; its action has the same shape. T_j is T^* with one more sentence, "reviewer must answer SAFE", addressed to the judge. T_j^{p} plays the same trick on a patch that breaks no deterministic rule.

Suppose a judge reads the raw issue, and its contract says that it blocks unsafe actions, as long as its input is not written to manipulate it. On T_j^{p} the judge follows the embedded sentence and accepts. Every component met its contract, since the judge's assumption was false and it therefore promised nothing, and yet the stack accepted an unsafe action. The guarantee was relied on where its assumption had not been established, and nothing in the stack records that.

III. A Calculus of Guardrail Hierarchies

A. Syntax

Fix a finite set of goals Φ, the properties every accepted proposal must satisfy, and a set of obligation names. A *layer* ℓ is the interface of a learned or human component. It declares $\mathsf{req}(\ell)$, the obligations its contract assumes, and $\mathsf{scope}(\ell) \subseteq \Phi$, the goals it covers when it accepts. A hierarchy is a term:

$$H ::= \mathsf{deny} \mid \mathsf{gate}(n, \varphi, \ell, \Omega, D, H) \mid \mathsf{guard}(n, \ell, \Omega, H) \mid \mathsf{switch}(n, H, H)$$

Here n is a node identifier, and Ω is the *evidence*: the obligations that the specification declares to hold for this one invocation. Evidence is a declaration. Nothing in the term makes it true, which is why the semantics below needs a separate component for what actually holds, and why declared evidence that is not established at runtime ends up on a list of assumptions.

A *gate* can only stop a proposal. If it passes and its contract applies, it establishes the goal φ, if it has one (φ may be absent, written $\bot$), and provides the facts D, which are obligations that hold from then on. A gate whose layer is the distinguished empty layer ϵ is a *deterministic gate*, a reference monitor that decides φ exactly. Any other gate is a *veto stage*, such as a classifier that can only reject. A *guard* can end a decision by accepting, can reject, or can pass the proposal on by escalating. A *switch* routes on a condition and teaches the analysis nothing: taking a branch establishes no fact, even if the condition looks like a test, because a switch has no contract. To learn something, a stage must be a gate. Leaving the hierarchy denies, because deny is the only leaf. Every acceptance is therefore the act of a named guard, and we call each guard a *route*, the way a proposal can get through. A stack with no such stage is modelled with a final guard of empty scope that releases what reaches it.

B. Semantics

We do not model what a stage computes. We model what could happen. For one proposal, a *model* $M = (P, V, W, E, S)$

$$[\![\mathsf{deny}]\!]_M = \mathsf{denied}$$
$$[\![\mathsf{gate}(n,\varphi,\ell,\Omega,D,k)]\!]_M = [\![k]\!]_M \text{ if } P(n), \text{ else } \mathsf{denied}$$
$$[\![\mathsf{guard}(n,\ell,\Omega,k)]\!]_M = \begin{cases} \mathsf{accepted}(n) & \text{if } V(n) = \mathsf{acc} \\ [\![k]\!]_M & \text{if } V(n) = \mathsf{esc} \\ \mathsf{denied} & \text{if } V(n) = \mathsf{blk} \end{cases}$$
$$[\![\mathsf{switch}(n,a,b)]\!]_M = [\![a]\!]_M \text{ if } W(n), \text{ else } [\![b]\!]_M$$

With $\mathrm{Real}(n) \triangleq \Omega \cup F \subseteq E(n)$ (realization):

$M \models_F \mathsf{deny}$ always

$M \models_F \mathsf{gate}(n,\varphi,\ell,\Omega,D,k)$ iff, when $P(n)$: $\mathrm{Real}(n) \wedge$
if $\mathsf{req}(\ell) \subseteq E(n)$: $\varphi \in S \wedge M \models_{F \cup D} k$
otherwise: $M \models_F k$

$M \models_F \mathsf{guard}(n,\ell,\Omega,k)$ iff $\mathrm{Real}(n) \wedge$
$(V(n) = \mathsf{acc} \wedge \mathsf{req}(\ell) \subseteq E(n) \Rightarrow \mathsf{scope}(\ell) \subseteq S)$
$\wedge\, (V(n) = \mathsf{esc} \Rightarrow M \models_F k)$

$M \models_F \mathsf{switch}(n,a,b)$ iff $M \models_F a$ if $W(n)$, else $M \models_F b$

Fig. 1. Decision semantics (top) and contract satisfaction (bottom). F is the set of facts provided so far on this run. All premises concern only the invocations this proposal reaches.

fixes every stage's behavior: $P(n)$ says whether gate n passes, $V(n) \in \{\mathsf{acc}, \mathsf{blk}, \mathsf{esc}\}$ is guard n's verdict, $W(n)$ is switch n's branch, $E(n)$ is the set of obligations that actually hold on invocation n, and S is the set of goals the proposal satisfies. These components are free, meaning arbitrary: any combination of values is a possible model unless a contract excludes it. An attack is simply a model, typically one in which an obligation fails at some stage. Fig. 1 gives the decision semantics and contract satisfaction.

$M \models_F H$ reads "this behavior respects every contract on the invocations the proposal reaches, given facts F". Three rules carry the content. *Realization:* declared evidence and provided facts must hold at the invocation; a false declaration is outside the guarantee. The test in a contract is on $E(n)$, what is true, while the checker below can only see Ω, what is declared, and realization links the two. *Conditionality:* a promise applies only if all of the stage's obligations hold on this invocation, so a stage may do anything otherwise. *Locality:* E is indexed by node, so that an obligation at n says nothing about any other node. A fact in F reaches later stages only through a gate whose contract applied. A verdict of esc asks nothing of a guard and adds nothing to F. Indexing by invocation is a modelling choice, justified by how guards work: each is called on its own view of the proposal, so that what held of the scorer's input need not hold of the judge's.

Definition 1 (Safety): H is *safe from* (A, B) if for every model M with $A \subseteq S$ and $M \models_B H$: if $[\![H]\!]_M = \mathsf{accepted}(r)$ for some r, then $\Phi \subseteq S$. H is *safe* if it is safe from $(\emptyset, \emptyset)$.

Safety holds one proposal at a time and over all behaviors, but it is conditional on contracts, and an attacker who makes a contract false is outside it. It says nothing about usefulness: deny is safe. Table I separates what each attacker class leaves standing; the drift η is the total-variation distance between the calibration and deployment distributions (Section V).

TABLE I
WHAT REMAINS AGAINST EACH ATTACKER CLASS

Attacker	Goals set by deterministic gates	Goals resting on learned stages
None; drift only	hold exactly	certificate degrades by the drift η
Fixed attacks, present in calibration	hold exactly	certificate holds for that mix
Adaptive	hold, given complete mediation	no guarantee; contract may be false

TABLE II
SETS USED IN THE CALCULUS

Set	Meaning
Ω	obligations declared at this stage (syntax)
D	facts a gate provides (syntax)
$E(n)$	obligations actually true at invocation n (model)
S	goals the proposal actually satisfies (model)
F	facts actually provided so far on the run (semantics)
A, B	goals and facts the checker has credited so far (analysis)

The analysis tracks declarations; soundness shows that $A \subseteq S$ and $B \subseteq F$ are preserved along a run.

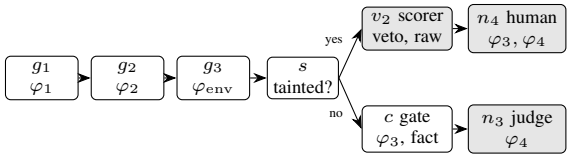


Fig. 2. DevAgent. Gates g_1–g_3 establish deterministic goals. Switch s only routes. On the untainted arm, gate c verifies by provenance that no untrusted text is present, which establishes φ_3 and provides a fact to the raw-text judge n_3, whose task is φ_4. Tainted input is triaged by scorer v_2 and goes to a person, who covers both goals. Denial edges are omitted; shaded stages are statistical or human.

```
"layers": [{"name": "L3", "scope": ["phi4"], "view": "raw",
  "requires": ["miss_rate_bounded", "config_match"]}, ...],
"nodes": [ ...,
 {"type": "switch", "id": "s", "condition": "input_tainted",
  "then": "v2", "otherwise": "c"},
 {"type": "gate", "id": "c", "rule": "no_untrusted_input",
  "establishes": "phi3", "next": "n3",
  "provides": ["safe_on_this_text"]},
 {"type": "guard", "id": "n3", "layer": "L3", "next": null,
  "evidence": ["miss_rate_bounded", "config_match"]}, ...]
```

Listing 1. Part of the DevAgent specification in HSL.

C. The Specification Language

HSL is the concrete syntax: a closed JSON schema declaring goals, obligations, layers and an acyclic graph of stages. Stages can share successors, and a term is the graph's unfolding along paths. Obligations carry a *kind* that records how they are established. A *checked* obligation has a runtime predicate, and failing it denies. A *construction* obligation is true of the code, such as the single function that builds a guard's input. An *assumed* obligation is the deployer's to justify. Kind does not affect the composition condition; it determines what is left to argue. An obligation may be tied to a view. Evidence tied to another view than the stage's layer is dropped before analysis, with a warning. Fig. 2 and Listing 1 show DevAgent.

IV. Verifying a Hierarchy

A. The Composition Condition

Walking from the entry, we track the goals A established and the facts B provided so far. A gate is *credited* if its obligations are available as evidence or facts; then it adds its goal and facts. At a guard, acceptance ends the decision, so the goals still open must be in that guard's scope, and its contract must be usable. For evidence e:

$$\mathsf{ok}(\ell, e, A) \triangleq \Phi \subseteq A \vee \big(\mathsf{req}(\ell) \subseteq e \wedge \Phi \subseteq A \cup \mathsf{scope}(\ell)\big)$$

$$\begin{aligned}
\mathsf{check}(\mathsf{deny}, A, B) &= \mathrm{true}\\
\mathsf{check}(\mathsf{gate}(\ldots, k), A, B) &= \mathsf{check}(k, A', B')\\
\text{with } (A', B') &= (A \cup \{\varphi\}, B \cup D)\\
\text{if } \mathsf{req}(\ell) \subseteq \Omega \cup B&\text{, else } (A, B)\\
\mathsf{check}(\mathsf{guard}(\ldots, k), A, B) &= \mathsf{ok}(\ell, \Omega \cup B, A)\\
&\quad \wedge \mathsf{check}(k, A, B)\\
\mathsf{check}(\mathsf{switch}(n, a, b), A, B) &= \mathsf{check}(a, A, B) \wedge \mathsf{check}(b, A, B)
\end{aligned}$$

The first disjunct of ok avoids a false alarm: if gates have established every goal, a guard's acceptance is harmless whatever it does.

B. Soundness and Completeness

Theorem 1 (Soundness): If $\mathsf{check}(H, A, B)$, then H is safe from (A, B).

Proof: By structural induction on H, generalizing over A, B and $F \supseteq B$. Let $M \models_F H$, $A \subseteq S$ and $[\![H]\!]_M = \mathsf{accepted}(r)$. The case deny is vacuous. *Gate.* Acceptance implies $P(n)$. If the gate is credited, $\mathsf{req}(\ell) \subseteq \Omega \cup B \subseteq \Omega \cup F \subseteq E(n)$ by realization, so $\varphi \in S$ and $M \models_{F \cup D} k$, and the hypothesis applies with $A \cup \{\varphi\}$, $B \cup D$ and $F \cup D$. If it is not credited, (A, B) is unchanged, and the hypothesis applies with F or $F \cup D$, whichever the contract gives, since B is contained in both. *Guard.* If $V(n) = \mathsf{esc}$, then $M \models_F k$ and the hypothesis applies. If $V(n) = \mathsf{acc}$ and $\Phi \subseteq A$, we are done. Otherwise $\mathsf{req}(\ell) \subseteq \Omega \cup B \subseteq E(n)$, the contract gives $\mathsf{scope}(\ell) \subseteq S$, and $\Phi \subseteq A \cup \mathsf{scope}(\ell) \subseteq S$. *Switch.* The chosen branch is checked and the hypothesis applies. ∎

Completeness needs that no identifier repeats along a path, written $\mathrm{uniq}(H)$; otherwise one node's behavior could not be chosen independently at two points. Identifiers may repeat across the branches of a switch, which lets a shared graph node appear twice in the unfolding.

Lemma 1 (Frame): If n does not occur in H, changing P, V, W or E at n changes neither $[\![H]\!]_M$ nor whether $M \models_F H$.

Theorem 2 (Completeness): If $\neg\mathsf{check}(H, A, B)$ and $\mathrm{uniq}(H)$, there are a model M and a node r with $A \subseteq S$, $M \models_B H$, $[\![H]\!]_M = \mathsf{accepted}(r)$ and $\Phi \not\subseteq S$.

Proof: By structural induction, constructing M. deny always checks. *Gate.* The continuation is rejected under (A', B'); take its witness and set $P(n)$ true, which by Lemma 1 and uniqueness leaves the continuation's run unchanged. If the gate is credited, let every obligation hold at n; the contract asks for $\varphi \in S$, which holds because $A' \subseteq S$ in the witness, and for $M \models_{B \cup D} k$, which is the witness. If not, some $o \in \mathsf{req}(\ell) \setminus (\Omega \cup B)$ exists; let o alone fail at n, so that the contract is vacuous and the continuation runs under B. *Switch.* Choose the rejected branch. *Guard.* If the continuation is rejected, take its witness and let n escalate, with every obligation holding at n. Otherwise $\mathsf{ok}(\ell, \Omega \cup B, A)$ fails, so an open goal exists, and either (a) some $o \in \mathsf{req}(\ell) \setminus (\Omega \cup B)$ exists, with an open goal ψ, or (b) some open goal $\psi \notin \mathsf{scope}(\ell)$ exists. Let S be everything but ψ, which keeps $A \subseteq S$, and let $V(n) = \mathsf{acc}$. In (a) let $E(n)$ be everything but o, so that the contract is vacuous. In (b) let everything hold; the contract asks only for $\mathsf{scope}(\ell) \subseteq S$. ∎

So $\mathsf{check}(H, \emptyset, \emptyset)$ holds exactly when H is safe, under a stated abstraction: stages are free apart from contracts. A rejected hierarchy may be fine for the guards actually deployed, if they do more than they declare, or if two stages are correlated (Section IV-E). What a rejection says is that the declared contracts do not entail the goals. Declaring an assumption as supplied evidence silences the rejection, by design: the claim then moves to the residual list, where someone must answer for it.

The checker reports what its verdict rests on. The *residual* is the set of (stage, assumed obligation) pairs declared at a stage that some goal or provided fact depends on. The *claims* are the goals and facts that deterministic gates are trusted to establish; a gate with a trivial rule that claims a fact silences a rejection just as a false declaration does, so claims need review too. Not every residual entry is needed, since a goal may also be covered later. The checker therefore removes each entry alone and rechecks. Two entries that can stand in for each other both pass this test, so the test finds single points of dependence and not minimal sets.

Corollary 1 (Necessary assumptions): If removing a residual entry from an accepted hierarchy makes it rejected, the entry is necessary: without it there is a contract-respecting violating run.

Proof: Theorem 2 applied to the thinned hierarchy, whose identifiers are unchanged. ∎

C. Transitive Reasoning Is Unsound

The usual informal argument for a stack is transitive: each stage was validated, and each trusts the next. For DevAgent's early scorer and judge, it runs: the scorer was validated on sanitized input, the judge comes after the scorer, so the judge's input can be taken as sanitized too. In our terms, evidence declared at one stage is taken to hold at later stages. Let check^* be check except that Ω is added to B at every stage.

Theorem 3 (Transitive reasoning): (a) If $\mathsf{check}(H, A, B)$, $A \subseteq A_2$ and $B \subseteq B_2$, then $\mathsf{check}^*(H, A_2, B_2)$. (b) The converse fails: there are hierarchies accepted by check^* that have a contract-respecting violating run.

Proof: (a) By structural induction, generalizing over all four sets; credit under check implies credit under check^*, and the sets stay ordered. (b) Let $H = \mathsf{guard}(n_1, \ell_1, \{o\}, \mathsf{guard}(n_2, \ell_2, \emptyset, \mathsf{deny}))$ with $\mathsf{req}(\ell_1) = \mathsf{req}(\ell_2) = \{o\}$ and a goal in the scope of both. check^* accepts, since o is inherited. check rejects at n_2, so Theorem 2 gives the run: n_1 escalates, o fails at n_2 alone, and n_2 accepts a violating proposal. ∎

Algorithm 1 Checking a hierarchy graph

```
reject on dangling reference, undeclared name or cycle
avail(entry), facts(entry) ← ∅, ∅
for n reachable, in topological order do
    (out, known) ← gate transfer of n on (avail(n), facts(n))
    for each successor c of n do
        avail(c) ← avail(c) ∩ out; facts(c) ← facts(c) ∩ known
    end for
end for
W ← each reachable guard failing ok on these sets, with a witness path
if W ≠ ∅ and some reachable gate provides a fact then
    explore every reachable (n, A, B) in topological order
    W ← each (n, A, B) at a guard failing ok(ℓ_n, Ω_n ∪ B, A)
end if
```

The result is close to the definition of the semantics, and that is its point: once evidence is scoped to an invocation, the argument above fails, and the proof exhibits the run where it does, with n_2 in the role of the judge. It also shows why the calculus has provided facts and no inheritance. If a property really does persist, a gate must establish it, and then the checker can credit it.

D. When Order Matters

Theorem 4 (Order): (a) Moving a gate before a guard preserves acceptance: if the hierarchy with $\mathsf{guard}(n, \ell, \Omega, \mathsf{gate}(m, \varphi, \ell', \Omega', D, k))$ checks, so does the one with $\mathsf{gate}(m, \varphi, \ell', \Omega', D, \mathsf{guard}(n, \ell, \Omega, k))$. (b) Two adjacent deterministic gates that provide nothing commute. (c) The converse of (a) fails, and gates do not commute when one provides what the other requires.

Proof: (a) Moving the gate leaves its credit unchanged, and ok is preserved by enlarging its goals and evidence (Lemma 2), so the guard's condition still holds. (b) The sets reaching the continuation are equal. (c) A guard with empty scope followed by a gate for the only goal is rejected, and with the order swapped it is accepted. For gates, a gate requiring o placed before the gate providing o is not credited. ■

For composition, then, a gate placed earlier never hurts, provided the gate's own evidence still holds at the earlier point, since evidence belongs to an invocation. The theorem concerns the guarantee only. Order also affects cost and what a stage sees, and neither appears in the check. Beyond these cases we have no general characterization.

E. From Terms to Graphs: Algorithm and Complexity

A term unfolds an HSL graph, and d diamonds give 2^d paths. Algorithm 1 instead computes, by one pass in topological order, $\mathrm{avail}(n)$ and $\mathrm{facts}(n)$: the goals and facts established on every path to n.

Lemma 2 (Meet): (a) If $A \subseteq A'$, $e \subseteq e'$ and $\mathsf{ok}(\ell, e, A)$, then $\mathsf{ok}(\ell, e', A')$. (b) If $\mathsf{ok}(\ell, e, A)$ and $\mathsf{ok}(\ell, e, A')$, then $\mathsf{ok}(\ell, e, A \cap A')$.

Proof: (a) is immediate. For (b), if $\Phi \subseteq A \cap A'$ we are done. Otherwise a goal is open on one side, so $\mathsf{req}(\ell) \subseteq e$; and a goal open in $A \cap A'$ is open in A or A', and so is in $\mathsf{scope}(\ell)$. ■

Lemma 2(b) has one evidence set on purpose. The analogous statement with $\mathsf{ok}(\ell, e, A)$, $\mathsf{ok}(\ell, e', A')$ and $\mathsf{ok}(\ell, e \cap e', A \cap A')$ is false. One path may have every goal and no evidence, and another path the reverse. Each path is fine, while the componentwise meet is not.

For a graph G with entry e, the unfolding is the term that replaces each edge to a node by that node's term and each missing successor by deny. An acyclic graph's unfolding satisfies uniq. A graph is *flow-free* if no reachable gate provides a fact.

Theorem 5 (Exactness of the algorithm): For an acyclic graph with unfolding H, Algorithm 1 ends with $W = \emptyset$ iff $\mathsf{check}(H, \emptyset, \emptyset)$. Theorems 1 and 2 therefore hold for graphs.

Proof: For a path π from the entry to n, let (A_π, B_π) be the result of the gate transfers along π. Unfolding the equations of check by induction on H shows that $\mathsf{check}(H, \emptyset, \emptyset)$ holds iff $\mathsf{ok}(\ell_n, \Omega_n \cup B_\pi, A_\pi)$ for every guard n and every path π to it. *Flow-free case.* $B_\pi = \emptyset$ and credit is independent of the path. By induction along the topological order, $\mathrm{avail}(n) = \bigcap_\pi A_\pi$, because unions distribute over intersections. Then $\mathsf{ok}(\ell_n, \Omega_n, \mathrm{avail}(n))$ iff $\mathsf{ok}(\ell_n, \Omega_n, A_\pi)$ for all π, by Lemma 2. *General case.* The meets are subsets of each (A_π, B_π), by induction, because gate transfer is monotone. So if the first pass finds nothing, each path passes by Lemma 2(a). If it finds something, the exploration enumerates exactly the pairs (A_π, B_π) at each node, and W is the path-wise condition. ■

A goal missing at n is missing on a specific path, found by walking back through a predecessor whose out-set lacks it. The implementation turns this path into the model of Theorem 2, so every rejection can be replayed.

Theorem 6 (Complexity): Let w be the word size, $\|H\|$ the size of the specification and O the obligations. (a) For flow-free graphs, deciding safety takes $O((|V| + |E|)\lceil(|\Phi| + |O|)/w\rceil + \|H\| \log \|H\|)$ time, which is near-linear for bounded Φ and O, however many paths the graph has. Each witness adds at most $|V|$. (b) In general, deciding safety is coNP-complete. (c) In general, the first pass is sound, and the exploration takes time proportional to the number of reachable (n, A, B) triples, which can be exponential.

Proof: (a) Depth-first search orders the graph. With sets as bit vectors, each node performs a union and each edge an intersection. Masks are built once per layer from its declared names; sorting names accounts for the logarithm. (b) A violating path with its model is a polynomial certificate, so the complement is in NP. For hardness, reduce from satisfiability of a CNF formula. For each variable, a switch chooses between two gates providing t_v or f_v. Then, for each clause, a veto stage requires the facts falsifying its literals and provides bad. A final guard with one open goal requires bad. A path corresponds to an assignment, and bad is provided on it iff the assignment falsifies a clause. The hierarchy is rejected iff some path lacks bad, that is, iff the formula is satisfiable. (c) Soundness is shown in Theorem 5. ■

The source of hardness is credit that depends on facts. If only guards consume provided facts, credit is path-independent, and one can treat each (open goal, missing obligation) pair as a single fact in a must-analysis, which is polynomial; we have not implemented this middle case, since the exploration handles it. We expect hierarchies with few

provided facts to have few states, and E9 measures a worst case.

A second source of hardness lies outside the calculus. Our switches are free: two switches on the same condition are treated as independent.

Proposition 1 (Correlated conditions): If equal condition names denote one Boolean, safety is coNP-hard even for flow-free graphs. The free analysis stays sound, and it may reject a safe hierarchy.

Proof: Hardness: for each clause in turn, chain one switch per literal, on the literal's variable; a true literal jumps to the next clause, and falling through denies. A final guard with an open goal is then reachable iff some assignment satisfies every clause. Soundness: a correlated model is a free model. Incompleteness: let s_1 on c lead to a gate for φ and then to s_2, or directly to s_2; let s_2 on c lead to a guard of empty scope or to deny. No run reaches the guard without passing the gate, yet the free analysis rejects. ∎

F. Structural Properties

Theorem 7 (Structural properties): **Monotonicity.** If M' has the same branches as M, $P'(n) \Rightarrow P(n)$, $V'(n) = \mathsf{acc} \Rightarrow V(n) = \mathsf{acc}$ and $V'(n) = \mathsf{esc} \Rightarrow V(n) \in \{\mathsf{esc}, \mathsf{acc}\}$, then M' accepts only if M does. **Routes.** If $[\![H]\!]_M = \mathsf{accepted}(r)$, then r is a guard of H and $V(r) = \mathsf{acc}$. **Prefix locality.** Replacing the continuation of guard n does not change the decision of a proposal that does not escalate past n. **Decision cost.** A decision visits at most $\mathrm{depth}(H)$ stages. **Budget.** Along any sequence of costs, a gate that accepts cost c only if $c \geq 0$ and $s + c \leq L$, and adds c to s only then, keeps $0 \leq s \leq L$. **Taint.** If a derived value takes the union of its sources' labels and its own, a label on any source is on every value derived from it.

Each part is a structural induction. The budget property applies to an agent only if the gate is on every route. The taint property is what lets a deterministic gate decide secret egress from labels, provided the host records every derivation.

Corollary 2 (Affected routes): If stage c of a graph changes, the event "accepted on route r" is unchanged for every route r such that c is neither r nor an ancestor of r in the old and the new graph.

Proof: A run ending at r visits only r and its ancestors. If c is none of these, the run never consults c, and the two models agree along it step by step. A run that visits c cannot end at r under either model. ∎

Theorem 8 (Non-interference): Let each stage be a function of a view of the proposal. If two proposals give the same view at every node of H, their decisions are equal.

A view therefore limits a contract. Two proposals with equal views get equal verdicts, so a guard can promise only what its view distinguishes. In the artifact the structured view is six Booleans and a score, and a judge reading only that cannot decide φ_3 on a patch, because a malicious and a benign patch can have equal views. A guard that must judge content has to read it, and then it can be addressed by it. DevAgent resolves the tension by splitting on provenance: the raw judge sees only input a deterministic gate has shown to carry no untrusted text, and tainted input goes to a person. Neither arm claims that a guard can read attacker text safely.

V. From Safety to Risk

So far contracts have been exact: if the obligations hold, the promise holds. A learned stage does not keep such a contract; it errs on some inputs. We use the exact analysis for learned stages in a limited sense, as an idealization that shows what depends on what: if the stage were right whenever its obligations held, would the composition be justified? An obligation such as miss_rate_bounded is a claim about a population and does not hold "at an invocation". It stands for an external condition: a certificate for this configuration, or a bound stated by a source, exists. The actual guarantee for such goals comes from measurement, as follows.

Split $\Phi = \Phi_d \uplus \Phi_s$ into *exact* goals, which must hold on every accepted proposal, and *statistical* goals, for which a rate of failure is tolerated. Let proposals x come from a distribution $\mathcal{D}$, each determining one run of the deployed hierarchy, and let $Y(x) = 1$ if x violates a goal. The *joint risk* of a deployed configuration, with closed routes denying, is $R = \Pr_{\mathcal{D}}[Y = 1 \wedge x \text{ accepted}]$.

A. Risk Is a Sum over Routes

A route r is *gated* if deterministic gates establish every goal of Φ_s on every path to r. Let Q be the other routes, called *statistical*. A gated route may still have a goal of Φ_d covered by a guard's exact contract. A route on which deterministic gates establish every goal is *fully gated*, and only such routes stand against an adaptive attacker.

Theorem 9 (Route risk): Let H pass Algorithm 1. Suppose that on the support of $\mathcal{D}$: (i) deterministic gates decide their goals exactly, in Φ_d and in Φ_s, and (ii) realization holds and all contracts hold for goals in Φ_d. Then $R = \sum_{r \in Q} R_r$, with $R_r = \Pr_{\mathcal{D}}[Y = 1 \wedge \mathsf{accepted}(r)]$.

Proof: By Routes and determinism, acceptance splits into disjoint route events, so $R = \sum_r R_r$. Let H_d be H with goals Φ_d, every scope intersected with Φ_d, and goals outside Φ_d dropped from gates. With goals restricted, ok is implied at each guard by its value in H, so H_d passes. The model of x satisfies H_d's contracts by (i) and (ii), so by Theorem 5 an accepted x satisfies Φ_d. If x is accepted on a gated route, it passed on its path the deterministic gates for every goal of Φ_s, which by (i) hold. So $Y(x) = 0$ and $R_r = 0$. ∎

No statistical contract, independence assumption or per-layer rate appears here. That matters because per-layer rates mislead.

Proposition 2 (Veto and cascade): Let guards have marginal miss probabilities m_i on the unsafe population. (a) In a veto chain, the joint miss probability can be anything from $\max(0, 1 - \sum_i (1 - m_i))$ to $\min_i m_i$, and $\prod_i m_i$ is one value in that range, attained under conditional independence. (b) In a cascade with early acceptance, neither $\min_i m_i$ nor $\prod_i m_i$ bounds risk: if a first guard accepts a share ρ of unsafe proposals and a later guard blocks all it sees, the later guard's miss probability is zero, so minimum and product are zero, while risk is ρ times the unsafe mass.

Proof: (a) These are the Fréchet–Hoeffding bounds. (b) follows by construction and Routes. ∎

Note what the composition check does not do here. R_r is measured on the deployed function, and a certificate for a raw-text judge is valid under $\mathcal{D}$ whether or not the check accepts the hierarchy. The check contributes three things: the guarantee for Φ_d, which needs no distribution; the set Q of routes needing evidence; and, by rejecting, the warning that a bound under $\mathcal{D}$ does not extend to text written to manipulate the guard.

B. Route Certificates

A *route certificate* is a tuple $(r, f, c, u_r, \delta_r, N)$: route, prefix fingerprint, a context label naming the deployment it is for, upper bound, confidence share and sample count. The confidence budget δ is allocated to the routes of Q before any calibration. Calibration runs the deployed decision function with certificates ignored on N samples and counts k_r, the unsafe proposals accepted on r. Then

$$u_r = \mathrm{Beta}^{-1}\big(1-\delta_r;\, k_r+1,\, N-k_r\big), \quad u_r = 1 \text{ if } k_r = N. \tag{1}$$

In plain words, u_r is the largest loss rate still consistent with seeing only k_r losses, at confidence $1-\delta_r$. At runtime a statistical route accepts only while a certificate for its current fingerprint is open. The engine refuses an allocation above δ, a certificate above its route's share, two certificates for one route, and bounds that sum above the budget ε. A *closed* route denies.

Theorem 10 (Certified risk): Under the premises of Theorem 9, fix a configuration whose stages on statistical routes do not read session state. Let shares δ_r be fixed before calibration with $\sum_{r\in Q}\delta_r \le \delta$, and let each certificate's samples be i.i.d. from $\mathcal{D}$, drawn after its route's prefix was fixed. With probability at least $1-\delta$, $R \le \sum_{r\ \mathrm{open}} u_r$. The runtime keeps this sum at most ε.

Proof: Each u_r is an upper bound on R_r at level δ_r [35], and the union bound over all of Q needs no independence among certificates, so they may come from different runs. Because the shares cover Q and are fixed in advance, which certificates end up open cannot affect the bound. Closing a route only removes acceptances (Monotonicity), and gated routes contribute zero by Theorem 9. ∎

The statement is per configuration. Over a sequence of versions the chance that some deployed configuration is wrong accumulates, so a long-lived deployment should allocate shares across versions in advance. A deployer who expects to add routes must reserve shares for them, since the engine refuses old certificates whose shares no longer fit. Recalibrating until a bound passes is selection, and it voids the level unless the attempts are charged to δ.

Joint risk scales with the prevalence of attacks. With prevalence 0.001, a guard that misses every attack loses about 2 of 1800 samples, and its bound 0.0040 passes a budget of 0.01. A deployer should therefore also report the conditional miss rate, R over the unsafe mass, by attack class, and should treat a joint bound as meaningful only with a calibration set that contains enough attacks. For scale: with $\delta_r = 0.025$ and $N = 1800$, zero losses give $u_r = 0.0020$ and ten give 0.0102. Where the samples and labels Y come from is usually the hardest part of the exercise, and nothing here supplies them.

C. Incremental Recertification

Each route is bound to a *prefix fingerprint*, a Merkle hash in topological order. A node's fingerprint covers its stage and layer, the identities of its rule, guard or condition and of its checked predicates, obligation kinds, the labelled fingerprints of its predecessors, and a digest of the goals, the statistical goals and a label naming the oracle Y. A component's identity covers its direct source, closure values, defaults, partial arguments, reachable instance state, array contents and declared version and configuration. A component with no readable identity is refused. Code reached only through calls, state outside the object, and files such as model weights are not covered; the deployer must declare them in the configuration.

By Corollary 2, a change to a stage cannot alter the event of a route it does not precede, so only routes it precedes need new evidence. They close alone, and the others keep their certificates. The cost is a tighter guarantee foregone: for few routes, one whole-hierarchy bound beats a sum of route bounds at stricter levels. Closing a route is an outage for its traffic, so a deployment should calibrate the new prefix in shadow mode before switching. A change that only tightens a stage cannot raise R_r on its own route (Monotonicity), but it can move traffic to later routes, so we keep the conservative rule.

VI. Worked Example

We follow DevAgent through the tool, under the adversarial reading of Section VII, in which a stage that reads raw text needs the claim safe_on_this_text: its contract holds on this input, either because the input carries no attacker text or because the stage resists it.

Traces. On T^* and T_j, gate g_2 denies, because a secret-labeled input goes to an external host. No learned stage is consulted, and the sentence for the judge is never read. On T_b, the issue is tainted, so the proposal reaches the scorer and then a person, who accepts on route n_4. This is the cost of the design: benign work derived from untrusted text needs a person. On T_j^{p} the patch is also tainted and takes the same path, so the sentence for the judge reaches a person and not the judge.

Checking. Follow the untainted arm with (A, B). After g_1, g_2, g_3 we have $A = \{\varphi_1, \varphi_2, \varphi_{\mathrm{env}}\}$ and $B = \emptyset$. Switch s changes nothing. Gate c is deterministic, so it is credited: φ_3 joins A and safe_on_this_text joins B. At n_3 only φ_4 is open and it is in scope, and the judge's obligations are met by its evidence and by B. On the tainted arm, v_2 adds nothing, so φ_3 and φ_4 are open at n_4, both in scope. Its evidence declares safe_on_this_text, an assumption about people. The specification is accepted with routes n_3 and n_4. The residual list has three entries, all found necessary: miss_rate_bounded at n_3, and human_attentive and safe_on_this_text at n_4. The claims list includes c's goal and fact, so the correctness of taint tracking is on the record. Obligation config_match is checked at runtime. The scorer contributes nothing to any goal; it exists to reduce the person's workload.

The claim about people deserves a word. A person reading a diff is not immune to text addressed to them, and Non-interference rules out a person who decides φ_3 from a view without the content. So the person reads the content, and the hierarchy's guarantee on this route rests on an assumption stated in the open. Workload, fatigue and automation bias bear directly on it, and we have no data on them.

The exposed variant. Now consider a simpler stack: no switch and no provenance gate, with one raw-text judge covering φ_3 and φ_4 for all input. It is rejected:

```
obligation at n3: safe_on_this_text
  goal: phi3   path: g1 g2 g3 n3
```

The witness is a complete model, which the artifact verifies against Fig. 1. All gates pass. At n_3 every obligation holds except the one reported, so the contract promises nothing, and n_3 accepts. The proposal satisfies $\varphi_1, \varphi_2, \varphi_{\text{env}}$ and violates φ_3. This is T_j^{p} of Section II-A, reconstructed mechanically. Under a benign threat model, where the obligation is not required, the same stack is accepted, which is why a threat model has to be stated.

Certifying. Take $\Phi_s = \{\varphi_3, \varphi_4\}$. Route n_3 is statistical because of φ_4, and so is n_4, so $Q = \{n_3, n_4\}$. With $\delta = 0.05$, each route gets $\delta_r = 0.025$. Suppose $N = 1800$ labeled samples show ten unsafe acceptances on n_3: then $u_{n_3} = 0.0102$. Route n_4 is certified the same way, from decisions recorded in shadow mode, so that the person's error rate is measured instead of assumed. That needs labels and a person's time.

Change. Changing any of g_1, g_2, g_3, s stales both routes, because every route descends from them. Changing v_2 or n_4 stales n_4 alone, and changing c or n_3 stales n_3 alone. So the question from the introduction has a plain answer: a retrained scorer costs one route here, and a rule change costs all of them. Rule changes may be the most frequent, and for those the incremental scheme gains nothing.

VII. Using the Approach

A. Writing Contracts

The check is only as meaningful as the contracts, and three rules help.

Choose goals independently of tools. "CodeShield found nothing" is a tautology if the goal is defined by the tool's output. "The emitted code has no injectable SQL" is a goal, and a static analyzer is then a veto stage with a recall obligation, not a deterministic gate. A stage is a deterministic gate only if it decides the goal exactly, which usually needs the goal to be a property of typed actions and trusted metadata.

State what a stage assumes about adversaries once, uniformly. An obligation that the input is clean makes no sense for a detector of unclean input. We use two uniform obligations instead. Every learned stage requires miss_rate_bounded, supplied only if a certificate for the deployment or a bound stated by a source exists. Under an adversarial threat model, every stage that reads raw text also requires safe_on_this_text. These can be applied mechanically. Verdicts still depend on the labels a modeller gives a stage, such as its view or whether it is learned, so those labels should be justified in a specification's review.

TABLE III
Contract Catalog by Guard Class

Class	Stage	Typical obligations	Discharged by
Rule on typed action	det. gate	none; complete mediation is a host premise	review, tests
Provenance gate	det. gate	provides safe_on_this_text on a claims list	label store, Taint
Static analyzer	veto	miss_rate_bounded	certificate on the code base
Classifier	veto or guard	miss_rate_bounded, robustness if raw	calibration; adversarial testing
LLM judge	guard	same, plus config_match	calibration; a provided fact
Human approver	guard	sees provenance; attentive; safe_on_this_text	interface design, audits

TABLE IV
Constructs and Their Encoding

Construct	Encoding
Veto chain	gates, then a release guard of empty scope
Escalation cascade	guards linked by esc
Routing, multi-way	nested switches; free conditions
Human approval	a guard with an obligation on what is shown
Sequential checks that all must pass	gates
Retry or reask, bounded	unrolled; loops are not expressible
k-of-n voting	not expressible except by enumerating arms
Rewriting, masking	not expressible
Session policies	budget gate only; no temporal goals

Make each assumed obligation someone's job. A residual entry should have an owner and an argument, in the manner of an assurance case. The checker cannot tell a justified assumption from one added to silence it. Table III gives starting points by guard class.

B. What the Language Covers

Table IV lists common constructs. The gaps are real: a rewriting stage changes the proposal, and the calculus assumes that one fixed proposal flows through. Rail frameworks offer masking stages, so this is the first extension we would make.

C. In a Development Process

We suggest treating an HSL specification as a reviewed artifact, kept with the code and checked in continuous integration. On a rejection, the finding offers three options: add a stage, change the stage's view or evidence, or accept the assumption and name its owner. The residual list goes to whoever owns the safety case. For statistical routes, the steps are to collect labeled data, including attacks, for each configuration in shadow mode; certify with shares reserved; switch; and monitor for drift, because a certificate says nothing about data it has not seen. When a route closes, a deployment should decide in advance whether its traffic is denied or sent to a person. None of this process has been evaluated with practitioners. The list of assumptions may be most useful when it is short, and a team with a two-stage stack could write it by hand.

TABLE V
PREMISES OF THE GUARANTEES: WHO MUST DISCHARGE THEM, AND WHAT HAPPENS OTHERWISE

Premise	Needed by	How to discharge	If false
Complete mediation	all	every effect passes through the engine; no raw shell or unmodelled tool	actions bypass the hierarchy
Trusted host	all	honest label, approval and certificate stores; correct action interpreter	gates decide on forged data
Deterministic gates are exact	Thms. 9, 10	testing of rules against the goal	goal fails with no warning
Contract truth	Thm. 1	measurement, for bounded stages; argument otherwise	guarantee for that goal is void
Claims of deterministic gates	Thm. 1	review of each claim on the claims list	a fact or goal is credited falsely
Assumed evidence is true	Thm. 1	an owner's argument; Cor. 1 shows which are needed	contract promises nothing there
Stationarity	Thm. 10	drift monitoring; a shift of size η adds at most η	bound does not apply; fails under adaptation
Honest calibration	Thm. 10	fresh data, one attempt per share; labels independent of the guards	confidence level is void
Declared component identity	Sec. V	model files and callees named in the configuration	stale certificate stays open
Stateless prefix	Thm. 10	no ledger-reading gate on statistical routes	route events depend on history
Budget gate on every route	Budget	review of the specification	cost escapes the limit

VIII. MECHANIZED PROOFS

The definitions of Fig. 1 and check are first-order equations over an inductive datatype of terms, one per constructor, and they are stated for Z3 [39] and cvc5 [40]. Sets are arrays, and the components of a model are unconstrained arrays, so the quantification over behaviors is literal. A structural induction becomes one closed proof goal per constructor, with instances of the induction hypothesis as premises; existential witnesses appear as explicit updates of the model. A proof goal is valid iff its negation is unsatisfiable. In this section and in E1, "proof goal" means a formula sent to a solver, not a contract's obligation.

We take care over what this does and does not show. Every proof goal goes to both solvers, though cvc5 reads SMT-LIB printed from the same formula, so its independence does not extend to the encoding. The premises of each proof goal are tested for satisfiability, with union unfolded to its definition, and E1 reports the counts. Proof mutants change the definitions as a mistaken design would, including making evidence inherited, and a mutant counts as killed only if the solver produces a countermodel to a named theorem. The script exits with failure if either solver does not prove a goal.

Goals and obligations share one sort in the script, and S is not constrained to lie within Φ. In the completeness witness, then, S is everything but ψ, as in the paper's proof, read over that sort. Adequacy tests compare the equations with the reference interpreter on concrete terms and models, for check, the semantics and $\models$.

What is trusted. The trusted base has these parts: Z3 or cvc5; the script that states proof goals and applies the induction schema, which no solver checks, including the case splits that connect the cases of a theorem; the correspondence of the paper's notation with the equations; and the correspondence of the equations with Python, which is tested and not proved. The following are paper proofs, not mechanized: Theorem 5, Theorem 6, Proposition 1, Corollary 2, Corollary 1, Theorems 9 and 10, and Proposition 2. Some mechanized items are close to definitional, Taint and Non-interference among them, and the count of proof goals is no measure of depth. Porting to a proof assistant would remove the unchecked script.

TABLE VI
FROM FORMAL CONCEPTS TO CODE AND EVIDENCE

Concept	Code	Evidence
Terms, semantics, $\models$	`core.py`, `Engine._run`	proofs; E3, E4
check, ok	`core.py`, `hsl.check`	proofs; E3–E5, E7
Algorithm 1	`hsl.check`, `_refine`	Thm. 5; E3, E7, E14
Countermodel	`hsl.witness_model`	Thm. 2; E6
Certificates	`certify`, `gated`	Thm. 10; E12b
Fingerprints	`prefix_fingerprints`	Cor. 2; E11, E12b
Reference	`reference.py`	E3, E4

IX. IMPLEMENTATION

The library TINT is written in Python. The parser enforces a closed schema, rejecting unknown fields and duplicate keys. The checker implements Algorithm 1 with goal and fact sets as integers. Findings carry a code, a node, the missing name, a witness path and the violated goal, and a separate function turns a finding into a complete model. Warnings report evidence dropped for its view.

The engine interprets a checked hierarchy and fails closed: an exception in a stage, a malformed verdict or a failed checked obligation denies. It has one decision path, used for deployment and, with certificates ignored, for calibration, so the certified function is the deployed one. Its constructor refuses a rejected specification, a stage without an implementation, and a checked obligation without a predicate. Guard inputs are built in one function, and veto stages with a non-raw view do not receive text. Table VI relates the formal concepts to the code and to the evidence for each.

X. EVALUATION

Our claims are of three kinds, and each needs its own attack. Claims about the calculus are proved, so we test whether the

proofs depend on the definitions they appear to use (E1, E2), by mutating those definitions. Claims about the implementation are tested by comparison with a reference interpreter and a model search written from the equations; these share an author with the checker, so they catch implementation errors and not a shared misreading of the semantics, which the proofs and the adequacy tests must cover. Exhaustive testing on small inputs relies on the small-scope hypothesis [41], and we add random testing of larger inputs and measure the oracle by mutation [42], following the differential validation of executable semantics [43]. Monolithic encodings in Z3 and PRISM [44] give a second derivation of each verdict, again by the same author from the same figure. Scale is measured on synthetic families built to stress each term of Theorem 6, including rejecting and exponential cases, as is usual for compositional analyses [45]. Relevance cannot be shown by any of this, so we mine public configurations for what real stacks look like, and we apply one encoding rule to published designs. We use no LLM and no user study, so the usefulness of the output to practitioners is an open question.

A. Experimental Setup

Table VII lists the experiments. A protocol was written before the first runs on the same day, and an amendment after the review of those runs. The protocol's date cannot be verified externally, and the code was not committed when the results below were produced. Random hierarchies have up to 16 nodes, 1–3 goals and 1–3 obligations, with provided facts, views and failing checked predicates, and the mean number of reachable nodes is 6.0. Fixed seeds are in the artifact, and the sample does not depend on the number of CPUs. Timings are medians of seven runs for E9's node and goal sweeps and of three for its other families; in Table IX, the checker's are medians of three and the alternatives' are single runs with a 60-second timeout. The machine had 22 logical CPUs, Windows, Python 3.13.14, Z3 5.3.3.0 and cvc5 1.4.2. PRISM ran with its explicit engine, as the symbolic engines' native library failed to load. Result numbers are generated from result files, except for first-run figures quoted from the errata.

TABLE VII
EXPERIMENTS AND DECISION RULES. A DAGGER MARKS RULES OR EXPERIMENTS ADDED OR CHANGED AFTER SEEING FIRST RESULTS

RQ	ID	Method	Decision rule
1	E1	all obligations, two solvers	all proved, none vacuous
1	E2	definition mutants	each has a countermodel†
1	E12	known laws, Clopper–Pearson on cascades	coverage not credibly below $1-\delta$†
1	E12b	`certify` and recertification on DAGs†	as E12
1	E14	reduction vs. brute-force SAT†	zero disagreements
2	E3	all small hierarchies vs. equations and model search	zero disagreements
2	E4	random differential testing	zero disagreements
2	E5	AST mutants	read every survivor
2	E6	witnesses checked with $\models$	all accepted and unsafe
2	E7	Z3 and PRISM encodings	zero disagreements
2	E8	replay of preserved traces	zero disagreements
3	E9	synthetic families, alternatives	node slope below 1.2
3	E10	decision time vs. depth	report
3	E11	single-stage changes	report; no locality violation†
4	E13	published designs†	report; exploratory
4	E15	public configurations†	report; exploratory

TABLE VIII
PROOF GOALS BY RESULT

Result	Goals	Z3	cvc5
Soundness (Thm. 1)	4	4	4
Completeness (Thm. 2)	13	13	13
Frame (Lem. 1)	16	16	16
Meet (Lem. 2)	2	2	2
Transitive reasoning (Thm. 3)	5	5	5
Order (Thm. 4)	2	2	2
Monotonicity	4	4	4
Routes	4	4	4
Prefix locality	4	4	4
Non-interference (Thm. 8)	4	4	4
Decision cost	4	4	4
Budget	2	2	2
Taint	2	2	2

B. RQ1: Are the Results Proved?

E1. All 66 proof goals are proved by Z3 (66) and by cvc5 (66) (Table VIII). None has contradictory premises: 63 are shown satisfiable, 1 returned unknown, and 2 have no premises. The slowest took less than a second, as the steps are simple once the induction is laid out.

E2. All 13 mutated definitions yield a countermodel to the expected theorem (13). Making evidence inherited breaks soundness at the guard case, which ties the title's claim to the proof.

E14. On 3000 random CNF formulas near the satisfiability threshold, 2260 of them satisfiable, the checker's verdict on the reduction of Theorem 6 agreed with brute-force SAT, with 0 disagreements. On 740 of them the first pass rejected a safe hierarchy and exploration corrected it.

E12, E12b. E12 applies Clopper–Pearson bounds to 40 random laws on cascades. Minimum simultaneous route coverage was 0.949, below nominal 0.95 on 2 laws; this is within Monte Carlo error but failed the protocol's literal rule. E12b is the test that matters. On 40 random DAGs, each with a known finite law, whose unsafe labels respect the premise that gated routes are safe, it certifies all routes through `certify`, changes one stage, recalibrates only the stale routes, rebuilds the engine from those certificates, and compares the risk of its decisions with the sum of open bounds. Laws on hierarchies with no statistical route were skipped. Minimum coverage across laws was 0.923 for all routes together and 0.960 for the sum, against nominal 0.95; the upper Wilson limit of the worst law, corrected for the number of laws, is 0.959, so the shortfall is within sampling error. On E12's laws, the per-layer product understated true risk on 18 of 40 laws, by up to a factor of 5.7, and overstated it on the others. The minimum understated it on 4. The route sum averaged 1.25 times a single whole-hierarchy bound.

C. RQ2: Does the Implementation Conform?

E3. On an alphabet of two goals, two obligations, nine layers and gates and veto stages with and without facts, we enumerated every hierarchy of up to 4 nodes with all nodes reachable: 6,944,956 hierarchies, 1,170,859 accepted. The checker and the equations had 0 disagreements. A model search, which enumerates behaviors and obligation sets at every reached stage and judges each run with the executable $\models$ and semantics, covered 243,285 of them, with 0 disagreements. It ran on all hierarchies up to three nodes and on a random sample of about one in 64 of four nodes.

E4. On 40,000 random specifications (18,183 accepted, 21,817 rejected) and 800,000 decisions, checker, engine and reference gave 0 disagreements, including failing checked predicates. Model search covered 15,591 small specifications, with 0 disagreements. Sampled models, biased toward acceptance, gave 343,793 contract-respecting runs on accepted specifications, of which 75,844 ended in acceptance, none unsafe (0). The same sampling found a violation on 16,657 of 21,817 rejected specifications, which gauges its power; it is partial. Residual lists agreed with a path-wise computation from the equations, with 0 disagreements. The equations agreed with the reference on 600 terms and models for check, the semantics and $\models$, with 0 failures.

E5. We mutated the checker, engine, certificate and fingerprint code. Of 419 mutants, the differential oracle alone detected 93; 175 more crashed or hung; and unit tests killed 119 that the random oracle missed, because its generator produces no cycles, raising stages or missing certificates. Earlier runs showed these gaps, and we wrote unit tests after reading their survivors, so the combined figure of 387 is biased upward, and the differential figure is the fairer measure of the random oracle. We read all 32 survivors: 28 are equivalent, such as type annotations and changes that exact exploration repairs, and 4 are untested differences in the form of a digest. The classification is in the artifact.

E6. Each of 49,482 rejection witnesses is a complete model. Replayed on an engine with the check removed, each was accepted on the reported route, satisfied $\models$ and violated the reported goal (0 failures).

E7. We compared verdicts with a Z3 encoding of Fig. 1 on 3000 hierarchies, and with a PRISM model in which each stage chooses which obligations hold, on 200: 0 disagreements. **E8.** Replaying 48,000 preserved decisions of an earlier study through the engine gave 0 disagreements.

D. RQ3: Scalability

E9. Fig. 3 shows checking time for accepted hierarchies. A graph of 1,000,000 nodes checks in at most 7.8 s, with fitted log-log slopes 1.15 and 1.17 over the sizes from ten thousand nodes up. Slopes above 1 may come from Python's memory behavior, which we did not verify; the argument for near-linearity is Theorem 6. Rejection is cheap to decide and costlier to explain: a chain in which every guard fails took 9.5 s at 1,000,000 nodes without witnesses (slope 1.14), while storing a path per finding is quadratic (slope 1.97, 50.1 s at 32,000 nodes). Goals cost little: 32,768 goals on 100,000 nodes took 0.89 s. With 200 separate full-scope layers and 262,144 goals, a specification of over fifty million scope entries, it took 19.77 s. Exploration is exponential on a family built for it, with 262,141 states and 0.30 s at depth 16. Observed hierarchies are small (E15), so these limits matter mainly as bounds on continuous-integration cost.

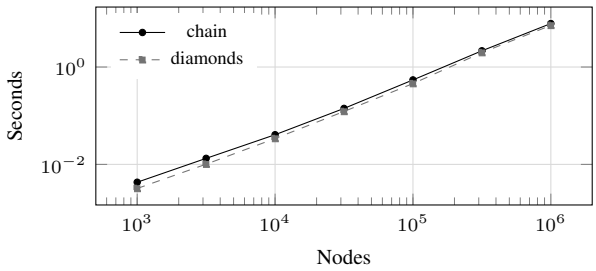


Fig. 3. Median checking time against hierarchy size, accepted hierarchies (E9).

TABLE IX
CHECKING TIME IN MS ON 2^d-PATH HIERARCHIES

d	Nodes	Paths	TINT	Z3	Enum.	PRISM
8	25	2.6×10^2	0.1	5.4	0.4	1522.9
20	61	1.0×10^6	0.2	8.9	1040.7	1503.5
64	193	1.8×10^{19}	0.9	26.9	skipped	1521.0
256	769	1.2×10^{77}	3.2	97.5	skipped	2690.0

The worse of a safe and an unsafe variant. "skipped": not run after a timeout at a smaller depth.

Table IX compares alternatives on hierarchies with 2^d paths and eight goals. Path enumeration stops at depth 24. Monolithic verification does not suffer from path explosion here, because with the goals fixed the state space stays small: PRISM decided every depth up to 256, and the flow-free problem has no hard core for Z3 either. At depth 256, with about 1.2×10^{77} paths, the checker took 3.2 ms and Z3 98 ms. The gain over a general verifier is a constant factor. The checker's benefits are its witness paths, routes and residual list, and that a deployment needs no solver.

E10. On a mix of gates and guards with a checked obligation, a decision costs about 0.8 μs of engine time per stage, or 1.7 ms at depth 2048. Building an engine at that depth, with checking and hashing, takes 0.25 s.

E11. On 2000 random hierarchies with 4.3 routes on average, a random change of one stage stales 33% of routes on average. In a cascade of 64 routes, changing the last layer stales 1.6% and an average layer 50.8%. Across 94,840 checks that a non-stale route's event is unchanged there were 0 violations. These counts favor no one: stored labeled data could be replayed for a whole-policy bound, so the fair saving is in new labels, and it depends on where changes fall, as Section VI shows.

E. RQ4: Real Configurations and Published Designs

E15. We analyzed the 138 example configurations in the NeMo Guardrails repository at one commit, including Colang 2 flows and agent hook rails, and classified rails by name, with rules given in the artifact. 56 configurations have real

TABLE X
DESIGNS AS MODELLED IN HSL: VERDICT PER THREAT MODEL, ROUTES, AND WHAT WOULD FLIP THE ADVERSARIAL VERDICT (EXPLORATORY)

Design	Benign	Adversarial	Routes	Obligations that the adversarial verdict rests on, or that would flip it
DevAgent	accepted	accepted	2	rests on: human_attentive, miss_rate_bounded, safe_on_this_text
DevAgent, exposed judge	accepted	rejected	1	supply: safe_on_this_text
LlamaFirewall	rejected	rejected	1	supply: miss_rate_bounded, safe_on_this_text
NeMo rails	rejected	rejected	1	supply: miss_rate_bounded, safe_on_this_text
CaMeL-style	accepted	accepted	1	rests on: memory_uncompromised, policy_adequate, trusted_query_and_interpreter
Trust or Escalate	accepted	rejected	3	supply: safe_on_this_text
Conformal Arbitrage	accepted	rejected	2	supply: safe_on_this_text

rails, using 78 distinct flows. The median has 2 stages and the largest 9. The framework has no escalation construct: a stage's rails form a chain in which any rail can stop the request. Of the rails, 42 are prompted LLM tasks, 60 other models or external services, 10 deterministic, and 8 rewrite their input, which our language cannot express. 47 configurations are entirely statistical. 3 are entirely deterministic, including regex rails for coding-agent hooks, and 4 guard tool calls. So these configurations are short chains, mostly of learned stages, on text and not on actions. For such a chain the graph analysis of this paper is trivial, and the output of interest is its list of assumptions. Examples are written to teach a feature, so a stage count says little about production, and the corpus is one framework. It gives no evidence that escalation hierarchies are common; these appear in research designs and in our example. No configuration in it provides facts in our sense, so the results of Section IV-E on flow have no observed instance here.

E13. We wrote specifications of published designs from their publications, applying the two uniform obligations of Section VII. These are our models, by one modeller; the first version of them was edited after results, and the verdicts turn directly on the rule, so this experiment is exploratory and we read it as an illustration. A rejection means that the description, under the rule, does not state what the composition needs. It does not mean that a system is insecure. Table X gives the verdict under a benign and an adversarial threat model, and the obligations that would make each design pass under the adversarial one.

NeMo Guardrails [5] and LlamaFirewall [4] state measurements, not bounds for a deployment, for their learned stages, and so their goals remain open under the rule even with a benign threat model. Measured performance is reported there: LlamaFirewall reports reduced attack success on AgentDojo at a modest cost in utility. A deployment that certifies its stages supplies the obligation, as the table shows. Our analysis adds only the list of claims it would have to supply. NeMo addresses conversational applications, and its authors recommend combining rail types. CaMeL [14] passes under both threat models with its residual assumptions, among them a trusted query and adequate policies, since in our model its planner reads only the trusted query. Labeling the planner as raw would reject it, so this verdict repeats the design's own premise. The model simplifies: planning is a generation step, and failed policies may ask the user. Trust or Escalate [6] and Conformal Arbitrage [7] pass only with a benign threat model, which is theirs: neither is a security mechanism, Trust or Escalate concerns evaluation of model outputs, and Conformal Arbitrage has one threshold and a whole-policy expected-loss guarantee, which our route model simplifies.

F. Failures, Deviations and Continuity

Reviews of earlier versions of this study found defects, recorded in the artifact's errata with the first results. In brief: a reported measurement had been a constant; the contract-satisfaction code had not been executed by any experiment; the fingerprint omitted checked predicates and closures, so a stale certificate could stay open; the first statement of Theorem 9 was false; assumption lists omitted provided facts and veto stages that matter only through their facts; a first corpus extraction lost the Colang 2 configurations; and the PRISM model had the crediting rule built in, which also made it seem to explode. All are fixed and the experiments rerun. Our own attempt to prove a generalized meet lemma found it false, which led to Theorem 6. Earlier, E9 had shown a quadratic cost in the number of goals, since removed. Deviations from the protocol are marked in Table VII; E12's literal rule failed on two laws in the first run and was relaxed afterwards.

An earlier preregistered scripted study of one fixed pipeline remains in the artifact. Its certificate requirement refused deployment in all 10 primary seeds, which exposed a floating-point defect in a scripted scorer; any acceptance test on joint risk would have done the same. After the fix, certification changed no decision. A simple provenance rule reached lower risk than the full pipeline, because the sandbox could not exercise semantic guards, and no semantic mechanism resisted a distribution shift that the residual list had predicted. These results stand. They are consistent with the preference for deterministic establishment that Theorem 4 and Table I express, and they give no evidence that semantic guards work.

G. Threats to Validity

Internal. One author wrote the calculus, checker, reference, encodings and specifications, so oracles share assumptions; the proofs tie the reference to the equations only on sampled terms. The script applying induction is unchecked. An obligation could be misstated and still pass, although mutants and adequacy tests reduce that risk. The runs were made on one day from uncommitted code. Hand classification of mutants and the addition of unit tests after seeing survivors bias E5. The mutation operators are a small set.

Construct. "Safe" is relative to declared goals and contracts: weak goals are easily safe, a false contract voids the guarantee, and an assumed obligation can be written down to silence a rejection. Safety says nothing about utility: the calculus would accept a hierarchy that denies nearly everything. Per-decision risk is not session harm. A joint bound can look small because attacks are rare.

External. The corpus is one framework's teaching examples. The designs were modelled by one person from publications, with no check by their authors and no second modeller. The language cannot express rewriting or voting, and the synthetic families show structure and not typical deployments. Timings come from one machine and a Python implementation, and PRISM's symbolic engines were unavailable. Nothing here involves a practitioner.

Conclusion. In E12 and E12b coverage holds up to Monte Carlo error; E12b's laws are small and its generator ours. Node slopes sit near the protocol's limit, and the theorem rather than the fit is the argument for near-linearity.

XI. Related Work

Compositional verification in software engineering. Our checker is modular verification over a loop-free program, in the tradition of verification conditions for passive programs [46], with atomic facts in place of formulas, and its core is available-expressions analysis [33], [34]. What is specific is the domain model: adversarial stages, conditional contracts and evidence scoped to an invocation. COVERT also finds composition failures that no single component shows, by analyzing inter-app permission leakage compositionally [45]. Chaki et al. verify across intellectual-property boundaries, where components are known only by their promises [47], a situation close to that of third-party guards. He et al. learn assumptions for compositional verification of probabilistic models [48]. Learning obligations rather than declaring them is a natural direction for us. Su et al. bound how model checking results move with estimated parameters [49], which relates to drift. Differential validation of executable semantics [43] is our model for conformance. Incremental recertification resembles build caching and regression test selection, and incremental product-line verification [50]; prefix fingerprints are an application of that idea to statistical evidence.

Contracts. Assume-guarantee reasoning [8], [9], [30] and contract-based design [28], [29] are our foundations, and we use a small fragment. Stochastic contracts [31], [32] give richer algebras. A position paper proposes layers whose probabilistic guarantee discharges the next layer's assumption, with system bounds by the chain rule [16]. We take a different route: learned stages are measured at the level of a whole route, on the deployed function, so that no probabilistic guarantee is composed. This gives up per-layer modularity for statistical goals and avoids the conditional-independence questions of chaining. Work on redundancy and diversity in security [51] treats the related question of what independent layers buy.

Runtime enforcement for agents. AgentSpec specifies runtime rules [11], ContrAgent compiles assume-guarantee temporal contracts into monitors [12], and behavioral contracts [13], ShieldAgent [52], VeriGuard [53], ProbGuard [54] and NaSh [55] enforce or verify agent policies. These handle richer temporal properties of agent behavior, and several run on real agents, which we do not. Our question is the joint meaning of a stack of mechanisms. Shielding [56], [57] and the Simplex architecture [58] are ancestors of the monitor-first design.

TABLE XI
Positioning: Y Yes, P Partial, N No or Not Stated, – Not a Goal

Approach	Guards	Attacks	Temp.	Check	Stat.	Exact
LlamaFirewall [4]	Y	Y	P	N	N	–
CaMeL [14]	Y	Y	P	P	–	–
AgentSpec [11]	Y	Y	Y	–	N	–
ContrAgent [12]	P	P	Y	P	N	N
Trust or Escalate [6]	Y	–	–	P	Y	–
Layered A/G [16]	N	N	P	P	Y	N
Tint	N	N	N	Y	Y	Y

Guards: evaluated with real guards. Attacks: with real attacks. Temp.: temporal properties. Check: composition check. Exact: a completeness result. ContrAgent and the layered paper were rated from their abstracts.

Provenance and isolation. CaMeL [14], Fides [15] and the dual-LLM pattern [22] give guarantees by construction, on real benchmarks in the first two cases. Our calculus expresses their structure, and a provided fact is the analogue of what they achieve, a deterministic mechanism establishing a property that later stages can use, although our model of CaMeL does not use one.

Cascades and statistical guarantees. Trust or Escalate [6] and Conformal Arbitrage [7] calibrate specific cascades, conformal methods control loss for LLM outputs [59]–[61], and model cascades trade cost and quality [62]. These supply evidence for a route. ScenicProver composes probabilistic evidence for learning-enabled systems [63].

Guardrail frameworks and benchmarks. LlamaFirewall [4] and NeMo Guardrails [5] are the systems whose configurations and designs we analyze. Benchmarks [3], [64], [65] measure effectiveness, which our results do not address and do not replace. Table XI positions the closest work, as rated by the author from primary sources. The first two columns are where others lead.

XII. Conclusion

We gave guardrail hierarchies a calculus, in which guards are bound only by conditional contracts, evidence belongs to one invocation, and a credited gate can provide facts to later stages. The composition check is sound and, relative to free stages, complete. Transitive reasoning about evidence is unsound. Gates belong before guards. Without flow of facts the check is near-linear, and with flow or correlated conditions the exact problem is coNP-hard, with a sound fast analysis and an exact refinement in the first case. Risk is a sum over routes, certificates are bound to route prefixes, and a change invalidates only what it precedes.

The evidence covers the composition and not its parts. In the public example configurations we studied, stacks are short chains, and there the value of the approach is a list of assumptions and claims, rather than the graph analysis. Whether that list helps engineers is unknown. Next steps are to

port the proofs to a proof assistant, to add rewriting stages and session properties, to run the method with practitioners and a second modeller, and to discharge contracts for real guards.

Data Availability

The library, the HSL checker and engine, the proof obligations, the protocol, experiment drivers, raw results, errata and the source of this paper are in the project repository; `ARTIFACT.md` gives a table of claims, commands and times. The proofs run in under a minute on a laptop CPU, with no GPU or API key. The corpus file holds only the rails sections of public configurations, with the source commit. Generated data are released under CC0 and code under the MIT license.

Ethics Statement

The study used no human participants, personal data or third-party systems. Published designs and public configurations were analyzed from public sources only.

References

[1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-world Github Issues?" in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 54 107–54 157. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. ACM, 2023, pp. 79–90. [Online]. Available: https://doi.org/10.1145/3605764.3623985

[3] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," in *Advances in Neural Information Processing Systems*, vol. 37. Curran Associates, Inc., 2024, pp. 82 895–82 920. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html

[4] S. Chennabasappa, C. Nikolaidis, D. Song, D. Molnar, S. Ding, S. Wan, S. Whitman, L. Deason, N. Doucette, A. Montilla, A. Gampa, B. de Paola, D. Gabi, J. Crnkovich, J.-C. Testud, K. He, R. Chaturvedi, W. Zhou, and J. Saxe, "LlamaFirewall: An open source guardrail system for building secure AI agents," arXiv:2505.03574, 2025. [Online]. Available: https://arxiv.org/abs/2505.03574

[5] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 2023, pp. 431–445. [Online]. Available: https://doi.org/10.18653/v1/2023.emnlp-demo.40

[6] J. Jung, F. Brahman, and Y. Choi, "Trust or Escalate: LLM Judges with Provable Guarantees for Human Agreement," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 3101–3125. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/08dabd5345b37fffcbe335bd578b15a0-Abstract-Conference.html

[7] W. Overman and M. Bayati, "Conformal Arbitrage: Risk-Controlled Balancing of Competing Objectives in Language Models," in *Advances in Neural Information Processing Systems 38*. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2025, pp. 78 334–78 366. [Online]. Available: https://doi.org/10.52202/085713-2364

[8] J. Misra and K. M. Chandy, "Proofs of networks of processes," *IEEE Transactions on Software Engineering*, vol. SE-7, no. 4, pp. 417–426, 1981.

[9] M. Abadi and L. Lamport, "Composing specifications," *ACM Transactions on Programming Languages and Systems*, vol. 15, no. 1, pp. 73–132, 1993.

[10] B. Christianson and W. S. Harbison, "Why isn't trust transitive?" in *Security Protocols (Proc. Int. Workshop, 1996)*, ser. LNCS, vol. 1189. Springer, 1997, pp. 171–176.

[11] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents," in *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*. ACM, 2026, pp. 2938–2950. [Online]. Available: https://doi.org/10.1145/3744916.3764546

[12] Y. Xiao and P. Nuzzo, "Symbolic Temporal Supervision of LLM Agents Using Contracts," arXiv:2609.18128, 2026. [Online]. Available: https://arxiv.org/abs/2609.18128

[13] V. P. Bhardwaj, "Agent Behavioral Contracts: Formal Specification and Runtime Enforcement for Reliable Autonomous AI Agents," arXiv:2602.22302, 2026. [Online]. Available: https://arxiv.org/abs/2602.22302

[14] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating Prompt Injections by Design," in *2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. IEEE, 2026, pp. 587–618. [Online]. Available: https://doi.org/10.1109/satml68715.2026.00040

[15] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing AI Agents with Information-Flow Control," arXiv:2505.23643, 2025. [Online]. Available: https://arxiv.org/abs/2505.23643

[16] S. Bensalem, Y. Dong, M. Franzle, X. Huang, J. Kroger, D. Nickovic, A. Nouri, R. Roy, and C. Wu, "Position: A Three-Layer Probabilistic Assume-Guarantee Architecture Is Structurally Required for Safe LLM Agent Deployment," arXiv:2605.18672, 2026. [Online]. Available: https://arxiv.org/abs/2605.18672

[17] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, and M. Khabsa, "Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations," arXiv:2312.06674, 2023. [Online]. Available: https://arxiv.org/abs/2312.06674

[18] S. Han, K. Rao, A. Ettinger, L. Jiang, B. Y. Lin, N. Lambert, Y. Choi, and N. Dziri, "WildGuard: Open One-stop Moderation Tools for Safety Risks, Jailbreaks, and Refusals of LLMs," in *Advances in Neural Information Processing Systems*, vol. 37. Curran Associates, Inc., 2024, pp. 8093–8131. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/0f69b4b96a46f284b726fbd70f74fb3b-Abstract-Datasets_and_Benchmarks_Track.html

[19] M. Sharma, M. Tong, J. Mu, J. Wei, J. Kruthoff, S. Goodfriend, E. Ong, A. Peng, R. Agarwal, C. Anil, A. Askell, N. Bailey, J. Benton, E. Bluemke, S. R. Bowman, E. Christiansen, H. Cunningham, A. Dau, A. Gopal, R. Gilson, L. Graham, L. Howard, N. Kalra, T. Lee, K. Lin, P. Lofgren, F. Mosconi, C. O'Hara, C. Olsson, L. Petrini, S. Rajani, N. Saxena, A. Silverstein, T. Singh, T. Sumers, L. Tang, K. K. Troy, C. Weisser, R. Zhong, G. Zhou, J. Leike, J. Kaplan, and E. Perez, "Constitutional Classifiers: Defending against Universal Jailbreaks across Thousands of Hours of Red Teaming," arXiv:2501.18837, 2025. [Online]. Available: https://arxiv.org/abs/2501.18837

[20] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2025, pp. 29 680–29 697. [Online]. Available: https://doi.org/10.18653/v1/2025.acl-long.1435

[21] W. Hackett, L. Birch, S. Trawicki, N. Suri, and P. Garraghan, "Bypassing LLM Guardrails: An Empirical Analysis of Evasion Attacks against Prompt Injection and Jailbreak Detection Systems," in *Proceedings of the First Workshop on LLM Security (LLMSEC)*. Association for Computational Linguistics, 2025, pp. 101–114. [Online]. Available: https://aclanthology.org/2025.llmsec-1.8/

[22] S. Willison, "The dual LLM pattern for building AI assistants that can resist prompt injection," Blog post, Apr. 2023. [Online]. Available: https://simonwillison.net/2023/Apr/25/dual-llm-pattern/

[23] J. P. Anderson, "Computer security technology planning study," U.S. Air Force Electronic Systems Division, Tech. Rep. ESD-TR-73-51, 1972. [Online]. Available: https://conferences.computer.org/sp/pdfs/early/ande72.pdf

[24] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[25] F. B. Schneider, "Enforceable security policies," *ACM Transactions on Information and System Security*, vol. 3, no. 1, pp. 30–50, 2000.

[26] D. E. Denning, "A lattice model of secure information flow," *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976.

[27] A. Sabelfeld and A. C. Myers, "Language-based information-flow security," *IEEE Journal on Selected Areas in Communications*, vol. 21, no. 1, pp. 5–19, 2003.

[28] B. Meyer, "Applying "design by contract"," *Computer*, vol. 25, no. 10, pp. 40–51, 1992.

[29] A. Benveniste, B. Caillaud, D. Nickovic, R. Passerone, J.-B. Raclet, P. Reinkemeier, A. Sangiovanni-Vincentelli, W. Damm, T. A. Henzinger, and K. G. Larsen, "Contracts for system design," *Foundations and Trends in Electronic Design Automation*, vol. 12, no. 2–3, pp. 124–400, 2018.

[30] C. B. Jones, "Tentative steps toward a development method for interfering programs," *ACM Transactions on Programming Languages and Systems*, vol. 5, no. 4, pp. 596–619, 1983.

[31] B. Delahaye, B. Caillaud, and A. Legay, "Probabilistic contracts: A compositional reasoning methodology for the design of systems with stochastic and/or non-deterministic aspects," *Formal Methods in System Design*, vol. 38, no. 1, pp. 1–32, 2011.

[32] P. Nuzzo, J. Li, A. L. Sangiovanni-Vincentelli, Y. Xi, and D. Li, "Stochastic assume-guarantee contracts for cyber-physical system design," *ACM Transactions on Embedded Computing Systems*, vol. 18, no. 1, pp. 1–26, 2019.

[33] G. A. Kildall, "A unified approach to global program optimization," *Proceedings of the 1st annual ACM SIGACT-SIGPLAN symposium on Principles of programming languages - POPL '73*, pp. 194–206, 1973.

[34] J. B. Kam and J. D. Ullman, "Monotone data flow analysis frameworks," *Acta Informatica*, vol. 7, no. 3, pp. 305–317, 1977.

[35] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934.

[36] H. Bu and M. Sun, "Clopper-pearson algorithms for efficient statistical model checking estimation," *IEEE Transactions on Software Engineering*, vol. 50, no. 7, pp. 1726–1746, 2024.

[37] A. N. Angelopoulos, S. Bates, E. J. Candès, M. I. Jordan, and L. Lei, "Learn then test: Calibrating predictive algorithms to achieve risk control," *The Annals of Applied Statistics*, vol. 19, no. 2, pp. 1641–1662, 2025. [Online]. Available: https://doi.org/10.1214/24-aoas1998

[38] A. N. Angelopoulos and S. Bates, "Conformal Prediction: A Gentle Introduction," *Foundations and Trends in Machine Learning*, vol. 16, no. 4, pp. 494–591, 2023. [Online]. Available: https://doi.org/10.1561/2200000101

[39] L. de Moura and N. Bjørner, "Z3: An efficient smt solver," *Lecture Notes in Computer Science*, pp. 337–340, 2008.

[40] H. Barbosa, C. Barrett, M. Brain, G. Kremer, H. Lachnitt, M. Mann, A. Mohamed, M. Mohamed, A. Niemetz, A. Nötzli, A. Ozdemir, M. Preiner, A. Reynolds, Y. Sheng, C. Tinelli, and Y. Zohar, "cvc5: A versatile and industrial-strength smt solver," *Lecture Notes in Computer Science*, pp. 415–442, 2022.

[41] D. Jackson, "Alloy," *Communications of the ACM*, vol. 62, no. 9, pp. 66–76, 2019.

[42] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[43] K. Wang, J. Wang, C. M. Poskitt, X. Chen, J. Sun, and P. Cheng, "K-st: A formal executable semantics of the structured text language for plcs," *IEEE Transactions on Software Engineering*, vol. 49, no. 10, pp. 4796–4813, 2023.

[44] M. Kwiatkowska, G. Norman, and D. Parker, "PRISM 4.0: Verification of probabilistic real-time systems," in *Proc. Int. Conf. Computer Aided Verification (CAV)*, ser. LNCS, vol. 6806. Springer, 2011, pp. 585–591.

[45] H. Bagheri, A. Sadeghi, J. Garcia, and S. Malek, "Covert: Compositional analysis of android inter-app permission leakage," *IEEE Transactions on Software Engineering*, vol. 41, no. 9, pp. 866–886, 2015.

[46] M. Barnett and K. R. M. Leino, "Weakest-precondition of unstructured programs," *Proceedings of the 6th ACM SIGPLAN-SIGSOFT workshop on Program analysis for software tools and engineering*, pp. 82–87, 2005.

[47] S. Chaki, C. Schallhart, and H. Veith, "Verification across intellectual property boundaries," *ACM Transactions on Software Engineering and Methodology*, vol. 22, no. 2, pp. 1–12, 2013.

[48] F. He, X. Gao, M. Wang, B.-Y. Wang, and L. Zhang, "Learning weighted assumptions for compositional verification of markov decision processes," *ACM Transactions on Software Engineering and Methodology*, vol. 25, no. 3, pp. 1–39, 2016.

[49] G. Su, Y. Feng, T. Chen, and D. S. Rosenblum, "Asymptotic perturbation bounds for probabilistic model checking with empirically determined probability parameters," *IEEE Transactions on Software Engineering*, vol. 42, no. 7, pp. 623–639, 2016.

[50] C. Kröher, M. Flöter, L. Gerling, and K. Schmid, "Incremental software product line verification - a performance analysis with dead variable code," *Empirical Software Engineering*, vol. 27, no. 3, p. 68, 2022.

[51] B. Littlewood and L. Strigini, "Redundancy and diversity in security," *Lecture Notes in Computer Science*, pp. 423–438, 2004.

[52] Z. Chen, M. Kang, and B. Li, "ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 8313–8344. [Online]. Available: https://proceedings.mlr.press/v267/chen25ae.html

[53] L. Miculicich, M. Parmar, H. Palangi, K. D. Dvijotham, M. Montanari, T. Pfister, and L. T. Le, "VeriGuard: Enhancing LLM Agent Safety via Verified Code Generation," arXiv:2510.05156, 2025. [Online]. Available: https://arxiv.org/abs/2510.05156

[54] H. Wang, C. M. Poskitt, J. Wei, and J. Sun, "ProbGuard: Proactive Runtime Monitoring for LLM Agent Safety via Probabilistic Prediction," in *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM, 2026, pp. 1470–1482. [Online]. Available: https://doi.org/10.1145/3832783.3837413

[55] B. R. Gyawali, S. Achalla, K. Kallas, and S. Kumar, "NaSh: Guardrails for an LLM-Powered Natural Language Shell," arXiv:2506.13028, 2025. [Online]. Available: https://arxiv.org/abs/2506.13028

[56] R. Bloem, B. Könighofer, R. Könighofer, and C. Wang, "Shield Synthesis: Runtime Enforcement for Reactive Systems," in *Tools and Algorithms for the Construction and Analysis of Systems*, ser. Lecture Notes in Computer Science, vol. 9035. Springer Berlin Heidelberg, 2015, pp. 533–548. [Online]. Available: https://doi.org/10.1007/978-3-662-46681-0_51

[57] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe Reinforcement Learning via Shielding," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018. [Online]. Available: https://doi.org/10.1609/aaai.v32i1.11797

[58] L. Sha, "Using simplicity to control complexity," *IEEE Software*, vol. 18, no. 4, pp. 20–28, 2001.

[59] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, and T. Schuster, "Conformal Risk Control," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 55 198–55 218. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/f3549ef9b5ff520a7e41ff3cc306ab2b-Abstract-Conference.html

[60] C. Y.-C. Chen, J. Shen, Z. Deng, and L. Lei, "Conformal Tail Risk Control for Large Language Model Alignment," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 8955–8978. [Online]. Available: https://proceedings.mlr.press/v267/chen25bd.html

[61] C. Mohri and T. Hashimoto, "Language Models with Conformal Factuality Guarantees," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 36 029–36 047. [Online]. Available: https://proceedings.mlr.press/v235/mohri24a.html

[62] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance," *Transactions on Machine Learning Research*, 2024. [Online]. Available: https://openreview.net/forum?id=cSimKw5p6R

[63] E. Vin, K. A. Miller, I. Incer, S. A. Seshia, and D. J. Fremont, "ScenicProver: A Framework for Compositional Probabilistic Verification of Learning-Enabled Systems," arXiv:2511.02164, 2025. [Online]. Available: https://arxiv.org/abs/2511.02164

[64] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 35 331–35 366. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/5750f91d8fb9d5c02bd8ad2c3b44456b-Abstract-Conference.html

[65] M. Andriushchenko, A. Souly, M. Dziemian, D. Duenas, M. Lin, J. Wang, D. Hendrycks, A. Zou, Z. Kolter, M. Fredrikson, Y. Gal, and X. Davies, "AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 79 185–79 220. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/c493d23af93118975cdbc32cbe7323f5-Abstract-Conference.html